

Industrial-Grade Vehicle Damage Detection System

Complete Implementation Plan

1. Executive Summary

Automating vehicle damage assessment accelerates insurance claims, enables instant repair estimates, and removes manual inspection overhead. A single monolithic detection model struggles to exceed 90% accuracy because it conflates two unrelated tasks — finding damage and identifying car parts. This plan specifies a **3-stage modular pipeline**, the data strategy needed to support it, the training/inference code, the AWS infrastructure to run it cost-effectively, and a 12-week delivery timeline.

Target: >90% accuracy (mAP@0.5:0.95) with near-zero false positives, deployed as a production API.

2. System Architecture: The 3-Stage Pipeline

Separating tasks prevents the network confusion that occurs when one model tries to learn damage *and* part geometry simultaneously. Each stage is independently trainable, testable, and replaceable.

Stage	Model	Type	Purpose
1	Gatekeeper	Binary CNN (ResNet50 / MobileNet)	Confirms a car is present and damaged; filters out irrelevant or clean images before expensive processing
2	Damage Localization	Mask R-CNN (Instance Segmentation)	Traces pixel-perfect outlines of scratches, dents, broken parts
3	Car Part Mapping	Mask R-CNN or YOLACT	Segments structural components (hood, bumper, fender, doors)

Orchestration ("Smart Overlap"): A Python script computes Intersection over Union (IoU) between Model 2's damage masks and Model 3's part masks, producing outputs like *"Severe Dent detected on the Left Front Fender."*

Why this architecture

- Pros:** highest precision (pixel-level area for cost estimation); modular — adding a new damage class only requires retraining Model 2.

- **Cons:** three sequential networks increase compute cost and inference latency; the overlap orchestration adds engineering complexity.

Alternatives considered

- **YOLOv8 (single-stage detection):** fast and lightweight, but bounding boxes are too imprecise for repair-cost estimation and struggle with overlapping damages.
 - **RT-DETR / transformer-based video tracking:** better temporal consistency, but requires video datasets and substantially higher labeling/storage cost. Not justified for a static-photo claims workflow.
-

3. Data Strategy

Requirement	Spec
Dataset volume	4,000-14,000+ high-resolution images
Per-class minimum	1,500-2,000 annotated instances per damage type
Annotation format	Polygon masks exported to COCO JSON
Public source option	VehiDE dataset (~13,945 images, 32,000+ annotated damage instances)

Labeling rules:

- Polygons must trace the exact outline of damage (required for Mask R-CNN, not just bounding boxes).
- Where a dent and scratch overlap in the same location, label both instances separately.
- **Negative mining:** include healthy, undamaged car-part images left intentionally unannotated (e.g. `/Undamaged_Cars/`). This teaches the model what "normal" looks like and sharply reduces false positives.

Augmentation: Random horizontal flip, color jitter, and rotation to simulate varied lighting/angles and prevent overfitting. Use the **Albumentations** library — it correctly transforms polygon masks alongside the image, which standard augmentation libraries do not.

Annotation tooling: CVAT for polygon drawing; Roboflow or MakeSense.ai for dataset/version management.

4. Technology Stack

Layer	Tool
Language	Python 3.8+
Deep learning framework	PyTorch (torchvision Mask R-CNN)
CV preprocessing	OpenCV, Pillow
Annotation	CVAT, Roboflow / MakeSense.ai
Serving	FastAPI or Flask
Containerization	Docker
Cloud	AWS (S3, EC2, ECS)

5. Core Implementation Code

5.1 Training script (shared by Model 2 and Model 3 — same architecture, different datasets)

```
python
```

```

import torch
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor
from torch.utils.data import DataLoader
from your_dataset_module import CustomCOCODataset

def get_model_instance_segmentation(num_classes):
    model = torchvision.models.detection.maskrcnn_resnet50_fpn(pretrained=True)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, hidden
    return model

def train_model():
    device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
    num_classes = 2 # background + 1 class (extend per damage/part type)
    model = get_model_instance_segmentation(num_classes).to(device)

    dataset_train = CustomCOCODataset(root='data/train', annotation='data/train')
    data_loader = DataLoader(dataset_train, batch_size=2, shuffle=True,
                              num_workers=4, collate_fn=lambda x: tuple(zip(*x)))

    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=0.01)
    lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.1)

    num_epochs = 10
    for epoch in range(num_epochs):
        model.train()
        for images, targets in data_loader:
            images = [img.to(device) for img in images]
            targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())
            optimizer.zero_grad()
            losses.backward()
            optimizer.step()
        lr_scheduler.step()
        print(f"Epoch: {epoch} | Loss: {losses.item()}")

if __name__ == "__main__":
    train_model()

```

5.2 Orchestration script (Smart Overlap / IoU stitching)

python

```
import torch
import numpy as np

def calculate_iou(mask1, mask2):
    intersection = np.logical_and(mask1, mask2).sum()
    union = np.logical_or(mask1, mask2).sum()
    return 0 if union == 0 else intersection / union

def orchestration_pipeline(image_tensor, model_damage, model_parts, iou_threshold):
    model_damage.eval()
    model_parts.eval()
    with torch.no_grad():
        damage_preds = model_damage([image_tensor])[0]
        part_preds = model_parts([image_tensor])[0]

    final_report = []
    for i, damage_mask in enumerate(damage_preds['masks']):
        d_mask_bin = (damage_mask[0].cpu().numpy() > 0.5).astype(np.uint8)
        damage_label = damage_preds['labels'][i].item()

        for j, part_mask in enumerate(part_preds['masks']):
            p_mask_bin = (part_mask[0].cpu().numpy() > 0.5).astype(np.uint8)
            part_label = part_preds['labels'][j].item()

            overlap_iou = calculate_iou(d_mask_bin, p_mask_bin)
            if overlap_iou > iou_threshold:
                final_report.append({
                    "damage_type": damage_label,
                    "car_part": part_label,
                    "overlap_score": overlap_iou
                })

    return final_report
```

5.3 Performance optimizations

- **Automatic Mixed Precision** (`(torch.cuda.amp)`): halves memory use, speeds up training, allows larger batch sizes.
- **DataLoader tuning**: `(pin_memory=True)`, `(num_workers)` matched to CPU core count, so the GPU is never idle waiting on I/O.
- **Gradient accumulation**: simulate a larger effective batch size when GPU memory forces small batches (e.g. `(optimizer.step())` every 4 batches).

- **Albumentations augmentation:** keeps polygon masks correctly aligned through rotation/flip transforms.

6. AWS Infrastructure (Cost-Optimized)

Concern	Decision	Rationale
Data storage	Standard Amazon S3 → copy to EBS gp3 at training start	FSx for Lustre / EBS io2 are unnecessary for a ~20-50GB dataset; gp3 throughput is sufficient
Training compute	EC2 G4dn (NVIDIA T4) on Spot Instances	Best price/performance for CV training; Spot saves 70-90% vs. on-demand
Training service	Native EC2, not SageMaker	Avoids ~20% managed-service premium
Fault tolerance	Checkpoint weights to S3 every 10 minutes	Spot interruptions resume from the last checkpoint with zero lost progress
Inference compute	AWS Graviton3 (c7g) via ECS, CPU-only	Models exported to ONNX Runtime / OpenVINO ; ~1.5s inference is acceptable for an async claims workflow and avoids 24/7 GPU costs
Scaling	ECS auto-scaling to zero during low-traffic hours	Pay only for compute actually used

7. Evaluation Standards

Metric	Applies to	Purpose
mAP@0.5:0.95	Models 2 & 3	Strict overlap-threshold metric proving pixel-level localization quality
Precision	All models	Confirms near-zero false positives
Recall	All models	Confirms true damage instances are not missed
Box Loss / Mask Loss	Models 2 & 3	Tracks boundary-drawing quality during training
Accuracy	Model 1	Standard binary classification check

8. End-to-End Workflow

Phase 1 — Development & Training

1. **Data acquisition:** 4,000–14,000+ images (public datasets, scraping, historical claims).
2. **Annotation:** binary labels for Model 1; COCO-JSON polygons for Models 2 & 3.
3. **Preprocessing/augmentation:** flips, color jitter, rotation.
4. **Infrastructure setup:** S3 + EBS gp3, G4dn Spot instances, Docker environments.
5. **Training:** transfer learning for all three models (ImageNet-pretrained backbones).
6. **Evaluation/tuning:** iterate on learning rate, batch size, anchors until mAP exceeds the 90% target.

Phase 2 — Production Inference

1. **Image ingestion:** user submits a photo via mobile app; resized/normalized (e.g. 1024×1024 for Mask R-CNN).
 2. **Gatekeeper (Model 1):** Car present? Damaged? → reject, "no damage," or proceed.
 3. **Parallel segmentation:** Model 2 (damage masks + confidence) and Model 3 (part masks + confidence) run concurrently.
 4. **Orchestrator:** IoU overlap between damage and part masks.
 5. **Output generation:** e.g. *"Severe Dent located on the Front Left Fender."*
 6. **Downstream processing:** result queried against the insurer's cost database for an automated repair estimate.
-

9. Architecture Reference: When to Use Which Detector

Use case	Recommended model
Panel-level identification ("Left Front Door damaged")	R-CNN family — bounding boxes sufficient, backend processing, no real-time constraint
Discrete, isolated damage (missing mirror, cracked headlight, punctured tire)	R-CNN family
Live smartphone scanning for instant quotes	YOLOACT — real-time (30+ FPS) segmentation
Scratch/scrape surface-area calculation for paint cost	YOLOACT — captures thin, irregular shapes that boxes can't
Shattered glass / hail damage with many overlapping small marks	YOLOACT

10. Implementation Timeline (8-12 Weeks)

Weeks	Milestone
1-4	Data acquisition, polygon labeling, augmentation pipeline
5	Cloud infrastructure setup (GPU provisioning, Docker, data loaders)
6	Model 1 (Gatekeeper) — transfer learning fine-tune
7-10	Models 2 & 3 (instance segmentation) — training and hyperparameter tuning to cross the 90% mAP threshold
11-12	Orchestration script, API deployment (FastAPI + Docker), user acceptance testing on unseen images

11. Pipeline Diagram

